CAUSALLY ESTIMATING THE SENSITIVITY OF NEURAL NLP MODELS TO SPURIOUS FEATURES

Anonymous authors
Paper under double-blind review

ABSTRACT

Recent work finds modern natural language processing (NLP) models relying on spurious features for prediction. Mitigating such effects is thus important. Despite this need, there is no quantitative measure to evaluate or compare the effects of different forms of spurious features in NLP. We address this gap in the literature by quantifying model sensitivity to spurious features with a causal estimand, dubbed CENT, which draws on the concept of *average treatment effect* from the causality literature. By conducting simulations with four prominent NLP models — TextRNN, BERT, RoBERTa and XLNet — we rank the models against their sensitivity to artificial injections of eight spurious features. We further hypothesize and validate that models that are more sensitive to a spurious feature will be less robust against perturbations with this feature during inference. Conversely, data augmentation with this feature improves robustness to similar perturbations. We find statistically significant inverse correlations between sensitivity and robustness, providing empirical support for our hypothesis. Our findings contribute to the interpretation of models and their robustness.

1 INTRODUCTION

Despite the success of deep neural models on many Natural Language Processing (NLP) tasks (Liu et al., 2016; Devlin et al., 2019; Liu et al., 2019b), recent work has discovered that these models rely excessively on spurious features, making the right predictions for the wrong reasons (Gururangan et al., 2018; McCoy et al., 2019; Wang & Culotta, 2020). Neural NLP models learn correlations but not causation from training data (Feder et al., 2021) and thus they are easily fooled by *spurious correlation*: prediction rules that work for the majority examples but do not hold in general (Tu et al., 2020). For example, BERT (Devlin et al., 2019) only achieves an accuracy less than 10% on a challenge test set HANS (McCoy et al., 2019) for MNLI (Williams et al., 2018) where spurious correlation disappears. Measuring sensitivity to spurious feature is thus important for a more principled evaluation of and control over neural NLP models (Lovering et al., 2020). It is crucial to avoid overfitting our models to spurious features, but *how* to evaluate the risk of spurious features remains an open question. Although a wide range of evaluation approaches for robust NLP have been proposed (Ribeiro et al., 2020; Morris et al., 2020; Tu et al., 2020; Goel et al., 2021; Wang et al., 2021), to the best of our knowledge, a quantitative measure to evaluate or compare the effect of different spurious features on NLP models has yet to be proposed.

Inspired by the concepts of Randomized Controlled Trial (RCT) and Average Treatment Effect (ATE) in Causal Inference (Rubin, 1974; Holland, 1986), we quantify model sensitivity to spurious features through simulations: ① randomly labelling a dataset, ② individually injecting them (as treatments) into examples of a particular pseudo class, and ③ using ATE to measure the ease with which the model learns each feature. We dub our proposed metric **C**ausal s**EN**si**T**ivity (**CENT**). We realize the injection of artificial spurious features with non-adversarial perturbations[1] (Moradi & Samwald, 2021) in this work. The core intuition for our method is to frame RCT as a spurious feature identification task and formalize the notion of sensitivity as a causal estimand based on ATE. We conduct experiments on four neural NLP models with eight different spurious features. Analysis results based on CENT reveal the deadliest spurious feature and the brittlest model, which contributes better model interpretation.

[1] We use "spurious feature" and "perturbation" interchangeably in this work.

It stands to reason that a model that is more sensitive to a spurious feature will be less robust against test-time perturbations of the same feature. We use CENT to validate this intuition. To improve performance under perturbation, it is a common practice to leverage data augmentation (Li & Specia, 2019; Min et al., 2020; Tan & Joty, 2021), and we find evidence that the improvement is also strongly correlated to the model's sensitivity. Combining these two findings, we further show that data augmentation is *only* more effective at improving robustness against spurious features that a model is more sensitive to. Our work contributes to the groundwork for the evaluation of spurious feature sensitivity in NLP, while also contributing to the interpretation of robustness and data augmentation.

Our main contributions are summarized as follows:

- We use the concept of average treatment effect to quantify sensitivity of NLP models to spurious features and conduct an empirical analysis on typical nerual NLP models.
- We validate the inverse correlation between sensitivity and robustness of models to spurious features, justifying our proposed approach to sensitivity estimation.
- We demonstrate a significant relationship between sensitivity and performance boost of data augmentation.

2 BACKGROUND

Recent work finds that NLP models are more sensitive to spurious features than target features (Warstadt et al., 2020; Lovering et al., 2020), but the term "sensitivity" still does not map to a formal, quantitative measure in standard statistical frameworks. Estimating the effect of spurious feature on models is challenging, because it is often difficult to fully decouple the effect of the target features from spurious features in practice (Lovering et al., 2020). In the language of causality, this is *"correlation is not causation"*, due to the confounding target feature. Motivated by theories of causal inference, we propose CENT, a novel metric for estimating model's sensitivity to spurious features. As a means for introducing our metric later, we now review background knowledge on causality.

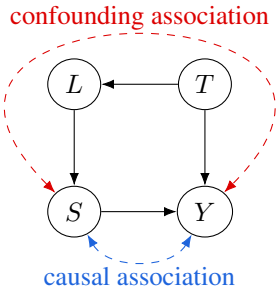
Causal Inference. The aim of causal inference is to investigate how a treatment T affects the outcome Y. Confounder X refers to a variable that influences both treatment T and outcome Y. For example, sleeping with shoes on (T) is strongly associated with waking up with a headache (Y), but they both have a common cause: drinking the night before (X) (Neal, 2020). In our work, we aim to study how a spurious feature (treatment) affects the model's prediction (outcome). However, the target features and other spurious features usually act as a confounder.

Causality offers solutions for two questions: 1) how to eliminate the spurious association and isolate the treatment's causal effect; and 2) how varying T affects Y, given both variables are causally-related (Liu et al., 2021). We will leverage both of these properties in our proposed method. Let us now introduce Randomized Controlled Trial and Average Treatment Effect as key concepts in answering the above two questions, respectively.

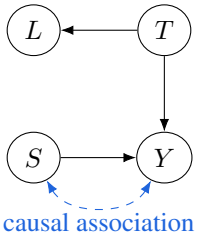
- **Randomized Controlled Trial (RCT).** In an RCT, each participant is randomly assigned to either the treatment group or the non-treatment group. In this way, the only difference between the two groups is the treatment they receive. Randomized experiments ideally guarantee that there is no confounding factor, and thus any observed association is actually causal. We operationalize RCT as a spurious feature classification task in Section 3.1.
- **Average Treatment Effect (ATE).** In Section 3.3, we apply ATE (Holland, 1986) as a measure of causal sensitivity. ATE is based on Individual Treatment Effect (ITE, Equation 1), which is the difference of the outcome with and without treatment.

$$ITE_i = Y_i(1) - Y_i(0) \tag{1}$$

  Here, $Y_i(1)$ is the outcome Y of individual i that receives treatment ($T = 1$), while $Y_i(0)$ is the opposite. In the above example, waking up with a headache ($Y = 1$) with shoes on ($T = 1$) means $Y_i(1) = 1$.



(a) Before randomization.



(b) After randomization.

Figure 1: Causal graph explanation for decoupling spurious feature and target feature with randomization. S is the spurious feature and T is the target feature. L is the original label and Y is the correctness of the predicted label.

We calculate the Average Treatment Effect (ATE) by taking an average over ITEs:

$$ATE = \mathbb{E}[Y(1)] - \mathbb{E}[Y(0)] \tag{2}$$

ATE quantifies how the outcome Y is expected to change if we modify the treatment T from 0 to 1. We provide specific definitions of ITE and ATE in Section 3.3.

3 METHOD

Setup and Terminology. We consider a binary sentential text classification problem with binary treatments (i.e., the spurious feature either exists or not). The training set is denoted as $D_{train} = \{(x_1, l_1), ..., (x_n, l_n)\}$, where x_i is the i-th example and $l_i \in \{0, 1\}$ is the corresponding label. We fit a model $f : (x; \theta) \mapsto \{0, 1\}$ with parameters θ on the training data. We assume that we have a transformation $g : (x; \beta) \rightarrow x^*$ that injects a specific type of spurious feature into an example x with parameters β and the perturbed example is x^*.

We cast sensitivity estimation as a spurious feature classification task, where a model is trained to identify the spurious feature in an example. Our proposed method consists of three steps, namely ① **random label assignment**, ② **spurious feature injection**, and ③ **causal estimation**. Below we detail the procedure and motivation for each step. We then summarize our estimation approach formally in Algorithm 1.

3.1 RANDOM LABEL ASSIGNMENT

We randomly assign pseudo labels to each training example regardless of its original label. Each data point has equal probability of being assigned to positive ($l' = 1$) or negative ($l' = 0$) pseudo label (i.e., the output of a coin toss). This results in a randomly labeled dataset $D'_{train} = \{(x_1; l'_1), ..., (x_n, l'_n)\}$, where $L' \sim Bernoulli(1, 0.5)$.

A Causal Explanation for Randomization. Spurious features naturally co-occur with target features in an example, making it a challenge to isolate the spurious feature's effect. If we did not assign random labels and simply injected spurious features into one of the *original* groups, there would be confounding target features that would prevent us from estimating the causal effect of the spurious feature. Figure 1a illustrates this scenario. Both spurious feature S and target feature T may affect the outcome Y[2], while the target feature is predictive of label L. Since we inject the spurious feature S into examples with the same label, S is decided by L. It therefore follows that T is a confounder of the effect of S on Y, resulting in non-causal association flowing along the path $S \leftarrow L \leftarrow T \rightarrow Y$. However, if we do randomize the labels, S no longer has any causal parents (i.e., incoming edges) (Figure 1b). This is because feature injection is purely random. Without the path represented by $S \leftarrow L$, all of the association that flows from S to Y is causal. As a result, we can directly calculate the causal effect from the observed outcomes (Section 3.3).

[2] Y is defined in Section 3.3

3.2 SPURIOUS FEATURE INJECTION

We apply the spurious transformation $g(\cdot)$ to each training example in one of the pseudo groups (e.g., $l' = 1$ in Algorithm 1)[3]. In this way, we create a spurious correlation between the injected feature and label (i.e., the feature occurrence is predictive of the label). We control the injection probability $p \in [0, 1]$, i.e., an example has a specific probability p of being injected with a spurious feature. This results in a perturbed training set $D'^{*}_{train} = \{(x^*_1, l'_1), ..., (x^*_n, l'_n)\}$, where the perturbed example x^*_i is:

$$Z \sim U(0,1), \forall i \in \{1, 2, ..., n\}, x^*_i = \begin{cases} g(x_i) & l'_i = 1 \wedge z < p, \\ x_i & \text{otherwise.} \end{cases} \tag{3}$$

Here Z is a random variable drawn from a uniform distribution $U(0, 1)$.

Criteria for Spurious Features. We inject spurious features into plain text by making non-adversarial, label-consistent perturbations. These perturbations can be automatically generated at scale. Note that our method does not require access to model-internal structure. We also assume that the injected spurious feature does not exist in original data. Not all perturbations in existing literature are suitable for our task. For example, a perturbation that swaps the gender word (i.e., female → male, male → female) will not result in a spurious feature since we cannot distinguish the perturbed text from an unperturbed one. In other words, the perturbation function $g(\cdot)$ should be *asymmetric*, such that $g(g(x)) \neq x$. We provide the list of spurious features we used in Appendix A.

3.3 CAUSAL ESTIMATION

Our randomization experiments allow us to discern causation from association and estimate the causal effect of injected spurious feature from test performance. We now train a model on the randomly labeled dataset with half of perturbed examples. Since the only difference between the two pseudo groups is the existence of the spurious feature, the model is trained to identify the spurious feature. The original test examples D_{test} are assigned random labels and become D'_{test}. We inject spurious feature into all of the test examples (injecton probability $p = 1$) in one pseudo group (e.g., $l' = 1$, as in Section 3.2) to produce a perturbed test set D'^{*}_{test}. Sensitivity is calculated as the difference of accuracies on D'^{*}_{test} and D'_{test}.

Identification of Causal Estimand for Sensitivity. In causality, the term "identification" refers to the process of moving from a causal estimand (ATE) to an equivalent statistical estimand. We show that the difference of accuracies on D'^{*}_{test} and D'_{test} is actually a causal estimand. We define the outcome Y of a test data point x_i as the correctness of the predicted label:

$$Y_i(0) \coloneqq \mathbf{1}_{\{f(x_i)=l'_i\}} \tag{4}$$

where $\mathbf{1}_{\{\cdot\}}$ is the indicator function. Similarly, the outcome Y of a perturbed test data point x^*_i is:

$$Y_i(1) \coloneqq \mathbf{1}_{\{f(x^*_i)=l'_i\}} \tag{5}$$

From Equation 1, the Individual Treatment Effect is $ITE_i = \mathbf{1}_{\{f(x^*_i)=l'_i\}} - \mathbf{1}_{\{f(x_i)=l'_i\}}$. We then take the average over all the perturbed test examples (half of the test set)[4]. This is our Average Treatment Effect (ATE). With Equation 2, we have:

$$\begin{aligned} ATE &= \mathbb{E}[Y(1)] - \mathbb{E}[Y(0)] \\ &= \mathbb{E}[\mathbf{1}_{\{f(x^*)=l'\}}] - \mathbb{E}[\mathbf{1}_{\{f(x)=l'\}}] \\ &= P(f(x^*) = l') - P(f(x) = l') \\ &= \mathcal{A}(f, g, p, D'^{*}_{test}) - \mathcal{A}(f, g, p, D'_{test}) \end{aligned} \tag{6}$$

where $\mathcal{A}(f, g, p, D)$ is the accuracy of model $f(\cdot)$ trained with feature $g(\cdot)$ at injection probability p on test set D. Therefore, we show that ATE is exactly the difference of accuracies on the perturbed

[3]Because the training data is randomly split into two pseudo groups, applying transformations to any one of the groups should yield same result. We assume that we always inject into the first group ($l' = 1$) hereafter.

[4]The other half of the test set ($l' = 0$) is left unperturbed, following the same procedure in Section 3.2. Model predictions will not change for unperturbed ones, resulting in ITEs with zero values. Therefore, we do not take them into account for ATE calculation.

Algorithm 1 Sensitivity Estimation

Input: training set $D_{train} = \{(x_1, l_1), ..., (x_n, l_n)\}$, test set $D_{test} = \{(x_{n+1}, l_{n+1}), ..., (x_{n+m}, l_{n+m})\}$, model $f : (x; \theta) \mapsto \{0, 1\}$, spurious perturbation $g : (x; \beta) \to x^*$, injection probability p
Output: ATE (sensitivity)

// ① random label assignment
Initialize an empty perturbed dataset D'
for i in $\{1, 2, ..., n + m\}$ **do**
  $l'_i \leftarrow randint[0, 1]$
  $D' \leftarrow D' \cup \{(x_i, l'_i)\}$
end for
// ② spurious feature injection
Initialize an empty perturbed and injected dataset D'^*
for i in $\{1, 2, ..., n + m\}$ **do**
  $z \leftarrow rand(0, 1)$
  $x^*_i \leftarrow x_i$
  if $l'_i = 1 \wedge z < p$ **then**
    $x^*_i \leftarrow g(x_i)$
  end if
  $D'^* \leftarrow D'^* \cup \{(x^*_i, l'_i)\}$
end for
// ③ causal estimation
$D'_{train}, D'_{test} \leftarrow D'[1 : n], D'[n+1 : n+m]$
$D'^*_{train}, D'^*_{test} \leftarrow D'^*[1 : n], D'^*[n + 1 : n + m]$
fit the model $f(\cdot)$ on D'^*_{train}
$\mathcal{A}(f, g, p, D'^*_{test}) \leftarrow f(\cdot)$ accuracy on D'^*_{test}
$\mathcal{A}(f, g, p, D'_{test}) \leftarrow f(\cdot)$ accuracy on D'_{test}
return $\mathcal{A}(f, g, p, D'^*_{test}) - \mathcal{A}(f, g, p, D'_{test})$

and unperturbed test sets with random labels. As a result, a higher ATE indicates a greater degree of sensitivity.

We discuss another means of identification of ATE in Appendix B.1, based on prediction probability. We compare between the probability-based and accuracy-based metrics there. We find that our accuracy-based metric yields better resolution, so we report this metric in this work.

Average over Different injection probabilities. We observe that the ATE-based sensitivity is dependent on injection probability p. For each model–feature pair, we obtain multiple ATE estimates by varying the injection probability (Figure 2). However, we expect that sensitivity of the model (as a concept) should be independent of injection probability. To this end, we use the $\log AUC$ (area under the curve in log scale) of the $p – ATE$ curve (Figure 2), termed as "average sensitivity", which summarizes the overall sensitivity across different injection probabilities $p_1, ..., p_t$:

$$\mathcal{A}(f, g, D) \coloneqq \log AUC(\{(p_i, \mathcal{A}(f, g, p_i, D'^*_{test}) - \mathcal{A}(f, g, p_i, D'_{test})) \mid i \in \{1, 2, ..., t\}\}) \quad (7)$$

We use $\log AUC$ rather than AUC because we empirically find that the sensitivity varies substantially between features when p is small, and a log scale can better capture this nuance. We also introduce sensitivity at a specific injection probability (Sensitivity @ p) as a summary metric and provide a comparison of this metric against $\log AUC$ in Appendix B.2.

4 Experiments

4.1 Estimating Sensitivity

Experimental Settings. To test sensitivity of different NLP models to various spurious features, we experiment with four modern and representative neural NLP models: TextRNN (Liu et al., 2016), BERT (Devlin et al., 2019), RoBERTa (Liu et al., 2019b) and XLNet (Yang et al., 2019). For TextRNN, we use the implementation by an open-source text classification toolkit NeuralClassifier (Liu et al., 2019a). For the other three pretrained models, we use the `bert-base-cased`, `roberta-base`, `xlnet-base-cased` versions from Hugging Face (Wolf et al., 2020), respectively. These two platforms support most of the common NLP models, thus facilitating extension studies of sensitivity of more models in future. We use a common binary text classification dataset — IMDB movie reviews (Pang & Lee, 2005) — as our testbed, which contains text labelled as positive or negative sentiment. We implement the injection of spurious features $g(\cdot)$ with two self-

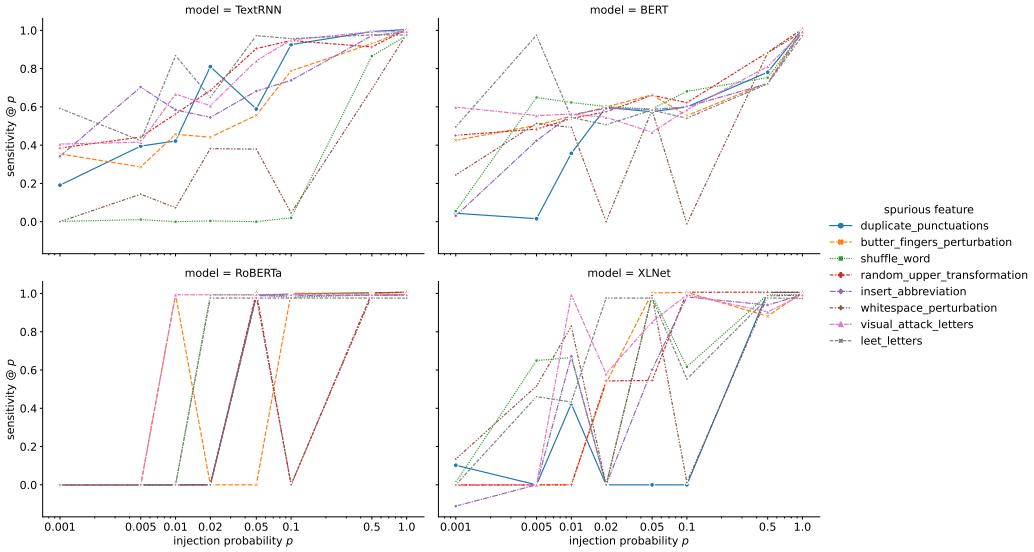


Figure 2: Sensitivity of four NLP models to eight spurious features, as a function of injection probability.

Spurious feature	RoBERTa	XLNet	BERT	TextRNN	Average over models
whitespace_perturbation	0.99	1.55	1.28	0.88	1.17
duplicate_punctuations	1.50	0.82	1.38	1.99	1.42
shuffle_word	<u>1.83</u>	1.83	1.81	0.60	1.52
random_upper_transformation	1.00	1.54	<u>1.90</u>	2.16	1.65
butter_fingers_perturbation	1.45	1.63	1.80	1.76	1.66
insert_abbreviation	1.48	1.48	1.61	2.08	1.67
visual_attack_letters	**2.14**	<u>1.90</u>	1.86	<u>2.18</u>	<u>2.02</u>
leet_letters	1.81	**1.95**	**1.98**	**2.36**	**2.03**
Average over features	1.53	1.59	1.70	1.75	1.64

Table 1: Average sensitivity of each model–feature pair ($\log AUC$ of corresponding curve in Figure 2). Rows and columns are sorted by average values over all features and models. The feature to which a model is most sensitive is highlighted in **bold** while the following one is <u>underlined</u>.

designed perturbations and six selected ones from the NL-Augmenter[5]. More details of spurious features/perturbations can be found in Appendix A. For injection probabilities, we choose 0.001, 0.005, 0.01, 0.02, 0.05, 0.10, 0.50, 1.00.

Results. Figure 2 shows model sensitivity as a function of injection probability for each of the eight spurious features. Sensitivity @ p generally increases as we increase the injection probability, and when we perturb all the examples (i.e., $p = 1.0$), every model can easily identify it well, resulting in the maximum sensitivity of 1.0. This shows that neural NLP models succumb to these spurious features eventually. At lower injection probabilities, some models still learn that spurious feature alone predicts the label. In fact, the major difference between different $p - ATE$ curves is the area of lower injection probabilities and this provides motivation for using $\log AUC$ instead of AUC as the summarization of sensitivity at different p (Section 3.3).

[5] https://github.com/GEM-benchmark/NL-Augmenter

Exp No.	Measurement	Label	Perturbation	Training Examples	Test Examples
0	Standard	original	$l \in \varnothing$	$(x_i, 0), (x_j, 1)$	$(x_i, 0), (x_j, 1)$
1	Sensitivity	random	$l' \in \{1'\}$	$(x_j, 0'), (x_i^*, 1')$	$(x_i^*, 1')$
2		random	$l' \in \{1'\}$	$(x_j, 0'), (x_i^*, 1')$	$(x_i, 1')$
3	Robustness	original	$l \in \{0, 1\}$	$(x_i, 0), (x_j, 1)$	$(x_i^*, 0), (x_j^*, 1)$
4	Data Augmentation	original	$l \in \{0, 1\}$	$(x_i, 0), (x_j, 1)$ $(x_i^*, 0), (x_j^*, 1)$	$(x_i^*, 0), (x_j^*, 1)$

Table 2: Example experiment settings for measuring sensitivity, robustness and improvement by data augmentation. We inject a spurious feature to an example if its label falls in the set of label(s) in "Perturbation" column. $\varnothing$ means no injection at all. Training/test examples are the expected input data, assuming we have only one negative $(x_i, 0)$ and positive $(x_j, 1)$ example in our original training/test set. l' is a random label and x^* is a perturbed example.

Table 1 shows the average sensitivity over all injection probabilities of each model–feature pair in Figure 2. Model-wise comparison[6] (across different columns in Table 1) shows that the non-pretrained model (TextRNN) is generally more sensitive than pretrained models (BERT, RoBERTa, XLNet). Our results are in line with recent findings that pretraining improves robustness to spurious correlations (Hendrycks et al., 2019; 2020; Tu et al., 2020). We also observe that RoBERTa is less sensitive than BERT, indicating that a larger pretraining corpus improves downstream robustness and confirms that RoBERTa is indeed robustly optimized (Liu et al., 2019b). Interestingly, the sensitivity of RoBERTa jumps from 0.0 to 1.0 in a sudden at a relatively small injection probability, instead of changing gradually as the injection probability increases. This shows that RoBERTa is good at generalizing from a small proportion of data with spurious feature. Tu et al. (2020) also present a similar finding that RoBERTa generalize better from minority patterns in the training set than BERT. We find that CENT complements to existing literature on model interpretations, providing a new perspective and a promising analysis tool.

Feature-wise comparison (across different rows) reveals the most sensitive spurious feature for each model. For example, all four models are highly sensitive to "visual_attack_letters" and "leet_letters" (Appendix A), likely due to their damaging effects on the tokenization process. Pretrained models are less sensitive to "white_space_perturbation" and "duplicate_punctuations", probably because they have little effect on the subword level tokenization, or they may have encountered similar noise in the pretraining corpora. The rank of feature sensitivity differs a lot between models (discrepancy of the underlined feature for each model in Table 1), indicating that a potential solution to a single spurious feature may not work for all models. Priority matters when dealing with spurious correlations. Analyzing the types of features a model is sensitive to can help us better understand what it can learn during training and enables fair comparison between different models and features.

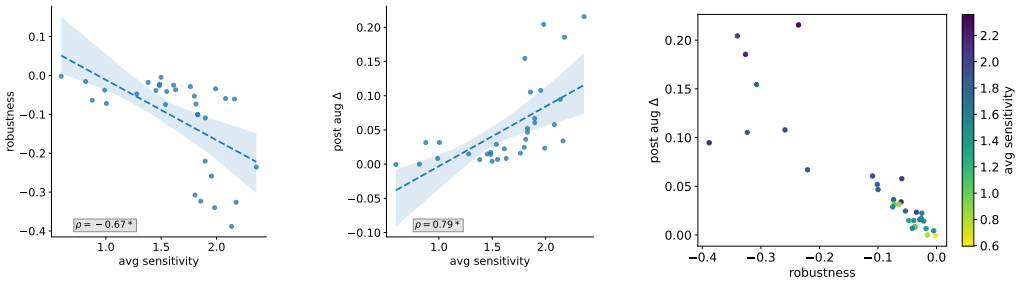
4.2 Investigating Sensitivity and Robustness.

Experimental Settings. Implementing spurious feature injections as perturbations allows us to apply the perturbations to test examples and measure the robustness of model to said perturbations as the decrease in accuracy. To this end, we design several experiment settings (Table 2). Experiment 0 in Table 2 is the standard learning setup, where we train and evaluate a model on the original dataset. Experiments 1 and 2 summarize the key points in sensitivity measurement (Section 3), including random label assignment and spurious feature injection. Specifically, Experiment 1 measures $\mathcal{A}(f, g, p, D'^{*}_{test})$, while Experiment 2 measures $\mathcal{A}(f, g, p, D'_{test})$ in Equation 6. We further use $\log AUC$ in Equation 7 to get rid of p. So the average sensitivity is:

$$\text{avg_sensitivity}(f, g, D) = \mathcal{A}(f, g, D). \tag{8}$$

Experiment 3 is related to robustness measurement, where we train a model on unperturbed dataset and test it on perturbed examples. We denote the test accuracy of a model $f(\cdot)$ on perturbed ex-

[6]We note that model-wise comparison is not fair across models with different numbers of parameters. Nevertheless, it is still instructive to compare models at their commonly-used size.



(a) Sensitivity vs. Robustness (b) Sensitivity vs. Post Aug Δ (c) Sen. vs. Robu. vs. Post Aug Δ

Figure 3: Linear regression plots of sensitivity vs. robustness vs. post data augmentation Δ against spurious features. Each point in the plots represents a model-feature pair. We define "avg sensitivity" as $\log AUC$ of the corresponding curve in Figure 2 (Equation 8), "robustness" as the performance drop on perturbed test set (Equation 9) and "post aug Δ" as the performance boost on perturbed test set (Equation 10). ρ is Spearman correlation. * indicates high significance (p-value < 0.001).

amples $g(\cdot)$ in Experiment 3 as $\mathcal{A}_3(f, g, D^*_{test})$. Similarly, the test accuracy in Experiment 0 is $\mathcal{A}_0(f, D_{test})$. Consequently, the robustness is calculated as the difference of test accuracies:

$$\text{robustness}(f, g, D) = \mathcal{A}_3(f, g, D^*_{test}) - \mathcal{A}_0(f, D_{test}). \tag{9}$$

Models usually suffer a performance drop when encountering perturbations, therefore the robustness is usually negative, where lower values indicate decreased robustness. Now we investigate the correlation between sensitivity and robustness, stated in the form of a hypothesis:

Hypothesis 1 (H1): *A model that is more sensitive to a spurious feature is less robust against the same spurious perturbation at test time.*

despite the fact that models encounter this feature during training in sensitivity estimation while they do not in robustness measurement.

To improve robust accuracy (Tu et al., 2020) (i.e., accuracy on the perturbed test set), it is a common practice to leverage data augmentation (Li & Specia, 2019; Min et al., 2020; Tan & Joty, 2021). We simulate the data augmentation process by appending perturbed data to the training set (Experiment 4 of Table 2). We calculate the improvement on performance after data augmentation as the difference of test accuracies:

$$\Delta_{\text{post_aug}}(f, g, D) = \mathcal{A}_4(f, g, D^*_{test}) - \mathcal{A}_3(f, g, D^*_{test}) \tag{10}$$

where $\mathcal{A}_4(f, g, D^*_{test})$ denotes the test accuracy of Experiment 4. $\Delta_{\text{post_aug}}(f, g, D)$ is the higher the better. We make another hypothesis:

Hypothesis 2 (H2): *A model that is more sensitive to a spurious feature experiences robustness gains with data augmentation along such a feature.*

We validate both Hypotheses 1 and 2 with experiments on various models and features described in Section 4.

Results. We observe a negative correlation between sensitivity (Equation 8) and robustness (Equation 9) in Figure 3a, validating Hypothesis 1, while Figure 3b quantifies the trend that data augmentation with a spurious feature the model is sensitive to improves robustness (Hypothesis 2). Both the correlations between 1) sensitivity and robustness and 2) sensitivity and data augmentation are strong (Spearman $\rho > 0.6$) and highly significant (p-value < 0.001), which firmly supports our hypothesis. We justify our proposed sensitivity metric by connecting it to robustness and validating the intuitive correlation between them. Our findings provide insight about when a model is less robust and when data augmentation works.

Figure 3c summarizes the information in Figure 3a and 3b. We observe that the average sensitivity decreases as robustness increases. This shows that the more sensitive a model is to a spurious feature, the greater the likelihood that its robustness can be improved through data augmentation along

this feature. We argue that this is not simply because there is more room for improvement by data augmentation. From a causal perspective, sensitivity acts as a common cause (confounder) for both robustness and data augmentation. This indicates a potential limitation of using data augmentation for improving robustness to spurious features (Jha et al., 2020): for insensitive features, data augmentation may be of little help. Approaches that go beyond simple data augmentation is required to combat such spurious features.

5 Related Work

Definitions of Sensitivity to Spurious Feature. In this paper, we quantify sensitivity as the ease with which a model learns a spurious feature classification task. However, we note that the term "sensitivity" is also used with other different meanings in the literature. *Sensitivity test* (Feder et al., 2021), e.g. *Counterfactually Augmented Data (CAD)* (Kaushik et al., 2019), evaluates the extent that models use spurious features to make predictions by injecting minimally label-flipping perturbations on target features. Gardner et al. (2021) also uses the term *sensitivity* in an informal way to describe the probability that a local edit that removes spurious features ("artifacts") changes the label.

Lovering et al. (2020) propose two metrics related to our definition of sensitivity: 1) the *extractability* of the feature from a model representation (operationalized as minimum description length, MDL) and 2) the model error on spuriously perturbed test examples (termed "*s-only error*"). However, they do not define sensitivity within causality framework[7]. The concept of sensitivity is defined more formally by Veitch et al. (2021), who term it *counterfactual invariance*. They propose distributional properties that a model not sensitive to spurious correlation should satisfy. Instead of properties, however, we propose a quantitative measure for sensitivity. We bridge the gap between causality and sensitivity by mathematically defining a causal estimand and devising a method (Algorithm 1) for estimating sensitivity.

Training with Random Labels. Pondenkandath et al. (2018); Maennel et al. (2020); Zhang et al. (2021) train deep neural networks (DNNs) on Computer Vision (CV) datasets with entirely random labels to study memorization, generalization, pretraining, and alignment. Though we similarly use random label assignment (Section 3.1) , our work is different from previous work in that 1) our insights behind randomization originate from the concept of Randomized Controlled Trial (RCT) in Causality; 2) we instead use randomization to study sensitivity to spurious features in NLP; 3) our labels are not purely random: they are correlated with the existence of spurious features.

Interpretation of Data Augmentation. Though data augmentation has been widely used in CV (Sato et al., 2015; DeVries & Taylor, 2017; Dwibedi et al., 2017) and NLP (Wang & Yang, 2015; Kobayashi, 2018; Wei & Zou, 2019), the underlying mechanism of its effectiveness remains underresearched. Recent studies aim to quantify intuitions of how data augmentation improves model generalization. Gontijo-Lopes et al. (2020) introduce affinity and diversity, and find a correlation between the two metrics and augmentation performance in image classification. In NLP, Kashefi & Hwa (2020) propose a KL-divergence–based metric to predict augmentation performance. Our proposed sensitivity metric CENT implies when data augmentation can help and thus act as a complement to this line of research.

6 Conclusion

Inspired by the concept of Average Treatment Effect (ATE) in Causal Inference, we causally quantify sensitivity of NLP models to spurious features. We validate the hypothesis that a model that is more sensitive to a particular spurious feature is less robust against the same spurious perturbation when encountered during inference. Additionally, we show data augmentation with the feature to improve its robustness to similar test-time perturbations. We hope CENT will encourage more research on spurious feature sensitivity and its implications for interpretability, in order to make *CENTs* of spurious correlation.

[7]Our work is also significantly different from Lovering et al. (2020) in that they further investigate the correlation between extractability and s-only error, while we instead investigate the correlation of spurious feature sensitivity with robustness and data augmentation.

7 ETHICS STATEMENT

Computing average sensitivity requires training a model for multiple times at different injection probabilities, which can be computationally-intensive if the sizes of the datasets and models are large. This can be a non-trivial problem for NLP practitioners with limited computational resources. We hope that our benchmark results of sensitivity of typical NLP models work as a reference for potential users. Collaboratively sharing results of such metrics on popular models in public fora can also help reduce duplicate investigation and coordinate efforts across teams.

To alleviate the computational efficiency issue of average sensitivity estimation, using sensitivity at selected injection probabilities may help at the cost of reduced precision (Appendix B.2). We are not alone in facing this issue: two similar metrics for evaluating spurious features, *extractability* and *s-only error* (Lovering et al., 2020) also require training the model repeatedly over the whole dataset. Therefore, finding an efficient proxy for average sensitivity is promising for more practical use of sensitivity in model interpretation.

REFERENCES

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, 2019.

Terrance DeVries and Graham W Taylor. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*, 2017.

Debidatta Dwibedi, Ishan Misra, and Martial Hebert. Cut, paste and learn: Surprisingly easy synthesis for instance detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 1310–1319. IEEE Computer Society, 2017.

Steffen Eger, Gözde Gül Şahin, Andreas Rücklé, Ji-Ung Lee, Claudia Schulz, Mohsen Mesgar, Krishnkant Swarnkar, Edwin Simpson, and Iryna Gurevych. Text processing like humans do: Visually attacking and shielding NLP systems. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 1634–1647, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1165. URL https://aclanthology.org/N19-1165.

Amir Feder, Katherine A Keith, Emaad Manzoor, Reid Pryzant, Dhanya Sridhar, Zach Wood-Doughty, Jacob Eisenstein, Justin Grimmer, Roi Reichart, Margaret E Roberts, et al. Causal inference in natural language processing: Estimation, prediction, interpretation and beyond. *arXiv preprint arXiv:2109.00725*, 2021.

Matt Gardner, William Merrill, Jesse Dodge, Matthew E Peters, Alexis Ross, Sameer Singh, and Noah Smith. Competency problems: On finding and removing artifacts in language data. *arXiv preprint arXiv:2104.08646*, 2021.

Karan Goel, Nazneen Fatema Rajani, Jesse Vig, Zachary Taschdjian, Mohit Bansal, and Christopher Ré. Robustness gym: Unifying the nlp evaluation landscape. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Demonstrations*, pp. 42–55, 2021.

Raphael Gontijo-Lopes, Sylvia Smullin, Ekin Dogus Cubuk, and Ethan Dyer. Tradeoffs in data augmentation: An empirical study. In *International Conference on Learning Representations*, 2020.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel Bowman, and Noah A Smith. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 107–112, 2018.

Dan Hendrycks, Kimin Lee, and Mantas Mazeika. Using pre-training can improve model robustness and uncertainty. In *International Conference on Machine Learning*, pp. 2712–2721. PMLR, 2019.

Dan Hendrycks, Xiaoyuan Liu, Eric Wallace, Adam Dziedzic, Rishabh Krishnan, and Dawn Song. Pretrained transformers improve out-of-distribution robustness. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 2744–2751, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.244. URL https://aclanthology.org/2020.acl-main.244.

Paul W Holland. Statistics and causal inference. *Journal of the American statistical Association*, 81 (396):945–960, 1986.

Rohan Jha, Charles Lovering, and Ellie Pavlick. Does data augmentation improve generalization in nlp? *arXiv preprint arXiv:2004.15012*, 2020.

Omid Kashefi and Rebecca Hwa. Quantifying the evaluation of heuristic methods for textual data augmentation. In *Proceedings of the Sixth Workshop on Noisy User-generated Text (W-NUT 2020)*, pp. 200–208, 2020.

Divyansh Kaushik, Eduard Hovy, and Zachary Lipton. Learning the difference that makes a difference with counterfactually-augmented data. In *International Conference on Learning Representations*, 2019.

Sosuke Kobayashi. Contextual augmentation: Data augmentation by words with paradigmatic relations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 452–457, 2018.

Zhenhao Li and Lucia Specia. Improving neural machine translation robustness via data augmentation: Beyond back-translation. In *Proceedings of the 5th Workshop on Noisy User-generated Text (W-NUT 2019)*, pp. 328–336, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-5543. URL https://aclanthology.org/D19-5543.

Liqun Liu, Funan Mu, Pengyu Li, Xin Mu, Jing Tang, Xingsheng Ai, Ran Fu, Lifeng Wang, and Xing Zhou. NeuralClassifier: An open-source neural hierarchical multi-label text classification toolkit. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 87–92, Florence, Italy, July 2019a. Association for Computational Linguistics. doi: 10.18653/v1/P19-3015. URL https://aclanthology.org/P19-3015.

Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. Recurrent neural network for text classification with multi-task learning. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, pp. 2873–2879, 2016.

Xiao Liu, Da Yin, Yansong Feng, Yuting Wu, and Dongyan Zhao. Everything has a cause: Leveraging causal inference in legal text analysis. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1928–1941, 2021.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019b.

Charles Lovering, Rohan Jha, Tal Linzen, and Ellie Pavlick. Predicting inductive biases of pretrained models. In *International Conference on Learning Representations*, 2020.

Hartmut Maennel, Ibrahim Alabdulmohsin, Ilya Tolstikhin, Robert JN Baldock, Olivier Bousquet, Sylvain Gelly, and Daniel Keysers. What do neural networks learn when trained with random labels? *arXiv preprint arXiv:2006.10455*, 2020.

Tom McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3428–3448, 2019.

Junghyun Min, R Thomas McCoy, Dipanjan Das, Emily Pitler, and Tal Linzen. Syntactic data augmentation increases robustness to inference heuristics. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 2339–2352, 2020.

Milad Moradi and Matthias Samwald. Evaluating the robustness of neural language models to input perturbations, 2021.

John Morris, Eli Lifland, Jin Yong Yoo, Jake Grigsby, Di Jin, and Yanjun Qi. TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 119–126, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.16. URL https://aclanthology.org/2020.emnlp-demos.16.

Brady Neal. Introduction to causal inference from a machine learning perspective. *Course Lecture Notes (draft)*, 2020. URL https://www.bradyneal.com/Introduction_to_Causal_Inference-Dec17_2020-Neal.pdf.

Bo Pang and Lillian Lee. Seeing stars: exploiting class relationships for sentiment categorization with respect to rating scales. In *Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics*, pp. 115–124, 2005.

Vinaychandran Pondenkandath, Michele Alberti, Sammer Puran, Rolf Ingold, and Marcus Liwicki. Leveraging random label memorization for unsupervised pre-training. *arXiv preprint arXiv:1811.01640*, 2018.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of nlp models with checklist. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912, 2020.

Donald B Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of educational Psychology*, 66(5):688, 1974.

Ikuro Sato, Hiroki Nishimura, and Kensuke Yokoi. Apac: Augmented pattern classification with neural networks. *arXiv preprint arXiv:1505.03229*, 2015.

Samson Tan and Shafiq Joty. Code-mixing on sesame street: Dawn of the adversarial polyglots. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 3596–3616, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.282. URL https://aclanthology.org/2021.naacl-main.282.

Lifu Tu, Garima Lalwani, Spandana Gella, and He He. An empirical study on robustness to spurious correlations using pre-trained language models. *Transactions of the Association for Computational Linguistics*, 8:621–633, 2020.

Victor Veitch, Alexander D'Amour, Steve Yadlowsky, and Jacob Eisenstein. Counterfactual invariance to spurious correlations: Why and how to pass stress tests. *arXiv preprint arXiv:2106.00545*, 2021.

William Yang Wang and Diyi Yang. That's so annoying!!!: A lexical and frame-semantic embedding based data augmentation approach to automatic categorization of annoying behaviors using #petpeeve tweets. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 2557–2563, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1306. URL https://aclanthology.org/D15-1306.

Xiao Wang, Qin Liu, Tao Gui, Qi Zhang, et al. Textflint: Unified multilingual robustness evaluation toolkit for natural language processing. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pp. 347–355, Online, aug 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-demo.41. URL https://aclanthology.org/2021.acl-demo.41.

Zhao Wang and Aron Culotta. Identifying spurious correlations for robust text classification. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Findings*, pp. 3431–3440, 2020.

Alex Warstadt, Yian Zhang, Xiaocheng Li, Haokun Liu, and Samuel R. Bowman. Learning which features matter: RoBERTa acquires a preference for linguistic generalizations (eventually). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 217–235, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.16. URL https://aclanthology.org/2020.emnlp-main.16.

Jason Wei and Kai Zou. Eda: Easy data augmentation techniques for boosting performance on text classification tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 6382–6388, 2019.

Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1112–1122, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1101. URL https://aclanthology.org/N18-1101.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL https://aclanthology.org/2020.emnlp-demos.6.

Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32, 2019.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3):107–115, 2021.

Spurious Feature	Example Sentence
None	His quiet and straightforward demeanor was rare then and would be today.
duplicate_punctuations	His quiet and straightforward demeanor was rare then and would be today..
butter_fingers_perturbation	His quiet and straightforward demeanor was rarw then and would be today.
shuffle_word	quiet would and was be and straightforward then demeanor His today. rare
random_upper_transformation	His quiEt and straightForwARd Demeanor was rare TheN and would be today.
insert_abbreviation	His quiet and straightforward demeanor wuz rare then and would b today.
whitespace_perturbation	His quiet and straightforward demean or wa s rare thenand would be today.
visual_attack_letters	Hiṣ qu̇iĕt a̋nd straa̧ghtf̓orwa̋rd̨ dɛmeanoŕ wa̅ṣ ra̍re tẖen and woul̩d þə tĉ d̲a᷄ỹ.
leet_letters	His qui3t and strai9htfor3ard d3m3an0r 3as rar3 t43n and 30uld 63 t0da4.

Figure 4: An example sentence injected with different spurious features.

A Details of Spurious Features

Figure 4 shows an example sentence injected with different spurious features. They are described in the following:

- **duplicate_punctuations:** We double the punctuations by appending a duplicate after each punctuation, e.g. "," → ",,".
- **butter_fingers_perturbation**[8]**:** This perturbation misspells some words with noise erupting from keyboard typos.
- **shuffle_word:** It randomly changes the order of word in the text (Moradi & Samwald, 2021).
- **random_upper_transformation**[9]**:** It randomly adds upper cased letters (Wei & Zou, 2019).
- **insert_abbreviation**[10]**:** It implements a rule system that encodes word sequences associated with the replaced abbreviations.
- **whitespace_perturbation**[11]**:** It randomly removes or adds whitespaces to text.
- **visual_attack_letters**[12]**:** This perturbation replaces letters with visually similar, but different, letters (Eger et al., 2019).
- **leet_letters**[13]**:** This perturbation replaces letters with leet, a common encoding used in gaming (Eger et al., 2019).

B Discussion

B.1 Another Identification of Causal Estimand For Sensitivity

In Section 3.3, we propose an accuracy-based identification of ATE. Now we discuss another probability-based identification and compare between them. We can also define the outcome Y

[8] https://github.com/GEM-benchmark/NL-Augmenter/tree/main/transformations/butter_fingers_perturbation

[9] https://github.com/GEM-benchmark/NL-Augmenter/tree/main/transformations/random_upper_transformation

[10] https://github.com/GEM-benchmark/NL-Augmenter/tree/main/transformations/insert_abbreviation

[11] https://github.com/GEM-benchmark/NL-Augmenter/tree/main/transformations/whitespace_perturbation

[12] https://github.com/GEM-benchmark/NL-Augmenter/tree/main/transformations/visual_attack_letters

[13] https://github.com/GEM-benchmark/NL-Augmenter/tree/main/transformations/leet_letters

of a test example x_i as the predicted probability of (pseudo) true label given by the trained model $f(\cdot)$:

$$Y_i(0) \coloneqq P_f(L' = l'_i \mid X = x_i) \in (0, 1) \tag{11}$$

Similarly, the performance outcome Y of a perturbed test data point x_i^* is:

$$Y_i(1) \coloneqq P_f(L' = l'_i \mid X = x_i^*) \in (0, 1) \tag{12}$$

For example, for a test example (x_i, l'_i) which receives treatment ($l'_i = 1$), the trained model $f(\cdot)$ predicts its label as 1 with only a small probability 0.1 before treatment (it has not been endowed with spurious feature yet), and 0.9 after treatment. So the Individual Treatment Effect (ITE, see Equation 1) of this example is calculated as $ITE_i = Y_i(1) - Y_i(0) = 0.9 - 0.1 = 0.8$. We then take an average over all the perturbed test examples (half of the test set)[14] as Average Treatment Effect (ATE, see Equation 2), which is exactly the sensitivity of a model to a spurious feature. To clarify, the two operands in Equation 2 are defined as follows:

$$\mathbb{E}[Y(1)] \coloneqq \mathcal{P}(f, g, p, D'^*_{test}) \tag{13}$$

It means the average predicted probability of (pseudo) true label given by the trained model $f(\cdot)$ on the perturbed test set D'^*_{test}.

$$\mathbb{E}[Y(0)] \coloneqq \mathcal{P}(f, g, p, D'_{test}) \tag{14}$$

Similarly, this is average predicted probability on the randomly labeled test set D'_{test}.

Notice that the accuracy-based definition of outcome Y (Equation 4) can also be written in a similar form to the probability-based one (Equation 11):

$$Y_i(0) \coloneqq \mathbf{1}_{\{f(x_i)=l'_i\}} = \mathbf{1}_{\{P_f(L'=l'_i|X=x_i)>0.5\}} \in \{0, 1\} \tag{15}$$

because the correctness of prediction is equal to whether the predicted probability of true (pseudo) label exceeds a certain thresholds, i.e., 0.5.

The major difference is that, accuracy-based ITE is a discrete variable falling in $\{-1, 0, 1\}$, while probability-based ITE is a continuous one ranging from -1 to 1. For example, if a model learns a spurious feature and thus changes its prediction from wrong (before spurious feature injection) to correct (after spurious feature injection), accuracy-based ITE will be $1 - 0 = 1$ while probability-based ITE will be less than 1. That is to say, accuracy-based ATE tends to vary more drastically than probability-based if inconsistent predictions occur more often, and thus can better capture the nuance of model's sensitivity. Empirically, we find that accuracy-based sensitivity has stronger correlation with robustness than probability-based one. Table 3 shows that Spearman ρ between probability-based average sensitivity and robustness is -0.63, weaker than -0.67 of accuracy-based one. As a result, we choose accuracy-based ATE as the primary measurement of sensitivity.

B.2 Investigating Sensitivity at a Specific injection probability

Inspired by Precision @ K in Information Retrieval (IR), we propose a similar metric dubbed Sensitivity @ p, which is the sensitivity of a model to a spurious feature at a specific injection probability p. We are primarily interested in whether a selected p can represent the sensitivity over different injection probabilities and correlates well with robustness and post data augmentation Δ.

We calculate the standard deviation (σ) of Sensitivity @ p and average sensitivity ($\log AUC$) over all model-feature pairs to measure how well it can distinguish between different models and features. Table 3 shows that average sensitivity is more diversified than all Sensitivity @ p and diversity (σ) peaks at $p = 0.1/0.02$ for accuracy-based/probability-based measurement. Accuracy-based Sensitivity @ p is generally more diversified across models and features than its counterpart.

To investigate the strength of the correlations, we also calculate Spearman ρ between accuracy-based/probability-based sensitivity @ p vs. average sensitivity/robustness/post data augmentation Δ over all model-feature pairs. Table 3 shows that average sensitivity has stronger correlations than all Sensitivity @ p. Correlations with both robustness and post data augmentation Δ peak at $p = 0.02/0.01$ for accuracy-based/probability-based measurement, and the correlations with average sensitivity (0.71*) are also strong at these injection probabilities.

[14]The other half of the test set ($l' = 0$) is left unperturbed, following the same procedure in Section 3.2. Therefore, we do not take them into account for ATE calculation.

p	Accuracy-based Sensitivity @ p				Probability-based Sensitivity @ p			
	σ	Avg Sen.	Robu.	Post Aug Δ	σ	Avg Sen.	Robu.	Post Aug Δ
Avg.	0.43	1.00*	-0.67*	0.79*	0.35	1.00*	-0.63*	0.73*
0.001	0.21	0.45*	-0.23	0.31	0.14	0.48*	-0.16	0.29
0.005	0.29	0.43*	-0.30	0.36*	0.23	0.74*	-0.40*	0.48*
0.01	0.33	0.50*	-0.38*	0.40*	0.23	0.71*	-0.44*	0.52*
0.02	0.35	0.71*	-0.52*	0.61*	0.25	0.76*	-0.43*	0.48*
0.05	0.30	0.29	-0.23	0.18	0.17	0.54*	-0.33	0.32
0.1	0.38	0.35	-0.10	0.15	0.23	0.56*	-0.22	0.21
0.5	0.10	-0.02	0.03	-0.11	0.03	0.10	-0.14	0.00
1.0	0.01	-0.11	0.20	-0.27	0.01	-0.08	0.21	-0.29

Table 3: Standard deviations (σ) of Sensitivity @ p and Spearman correlations between accuracy-based/probability-based sensitivity @ p vs. average sensitivity/robustness/post data augmentation Δ over all model-feature pairs. * indicates significance (p-value < 0.05).

Overall, Sensitivity @ p with higher standard deviation correlates better with average sensitivity, robustness and post data augmentation Δ. Our analysis shows that if p is carefully selected by σ, Sensitivity @ p is also a promising metric, though not as accurate as average sensitivity. One advantage of Sensitivity @ p over average sensitivity is that it costs less time to obtain sensitivity at a single injection probability. We plan to explore other efficient proxies of average sensitivity in future.